



MÓDULO 1 · T03

# Componentes y Props en React + TypeScript

 *Reutilización y tipado con TypeScript*

**Carlos Rodríguez Contreras** · GitHub: [karrocon](#)

# ¿Qué aprenderás hoy?

En esta sesión construirás una base sólida sobre cómo estructurar componentes reutilizables con tipado estricto en TypeScript. Al terminar, serás capaz de aplicar estos patrones en proyectos reales.

1

## Componentes funcionales

Crear componentes reutilizables con una API de props clara y bien tipada.

2

## Props con TypeScript

Definir interfaces y tipos estrictos para evitar errores en tiempo de compilación.

3

## Uniones de tipos

Usar `type TipoPokemon = 'fuego' | 'agua' | ...` para valores acotados.

4

## Composición

Aplicar el principio de composición combinando componentes simples en estructuras complejas.

# Anatomía de un componente

Todo componente bien diseñado en React + TypeScript sigue una estructura clara: primero se definen los tipos de las props, y luego se implementa el componente que las consume. Esta separación facilita la lectura y el mantenimiento del código.

```
// 1. Tipos de props
type PokemonCardProps = {
  nombre: string;
  thumbnail: string;
  tipo: TipoPokemon;
  onPress?: () => void; // opcional
};

// 2. Componente
export default function PokemonCard({
  nombre,
  thumbnail,
  tipo,
  onPress
}: PokemonCardProps) {
  return ( /* JSX */ );
}
```

- ① Separar los tipos del componente mejora la legibilidad y permite reutilizar los tipos en otros ficheros del proyecto.



# Uniones de tipos

## ¿Por qué usar uniones?

En lugar de un `string` genérico que acepta cualquier valor, una unión de tipos restringe los valores posibles al conjunto exacto que tu lógica espera.

TypeScript rechaza en tiempo de compilación cualquier valor que no esté en la unión – ¡no hace falta esperar a que falle en producción!

- ✓ El compilador te avisa si escribes `'Fuego'` en vez de `'fuego'` – tipo seguro al 100%.

## Los 18 tipos Pokémon

```
type TipoPokemon =  
  | 'normal' | 'fuego'  
  | 'agua' | 'planta'  
  | 'eléctrico' | 'hielo'  
  | 'lucha' | 'veneno'  
  | 'tierra' | 'volador'  
  | 'psíquico' | 'bicho'  
  | 'roca' | 'fantasma'  
  | 'dragón' | 'siniestro'  
  | 'acero' | 'hada';
```

# Diccionario de colores tipado

El tipo utilitario `Record<K, V>` de TypeScript crea un objeto cuyas claves son exactamente los valores de `TipoPokemon` y cuyos valores son `string`. Si olvidas un tipo o escribes una clave incorrecta, el compilador te lo indica al instante.

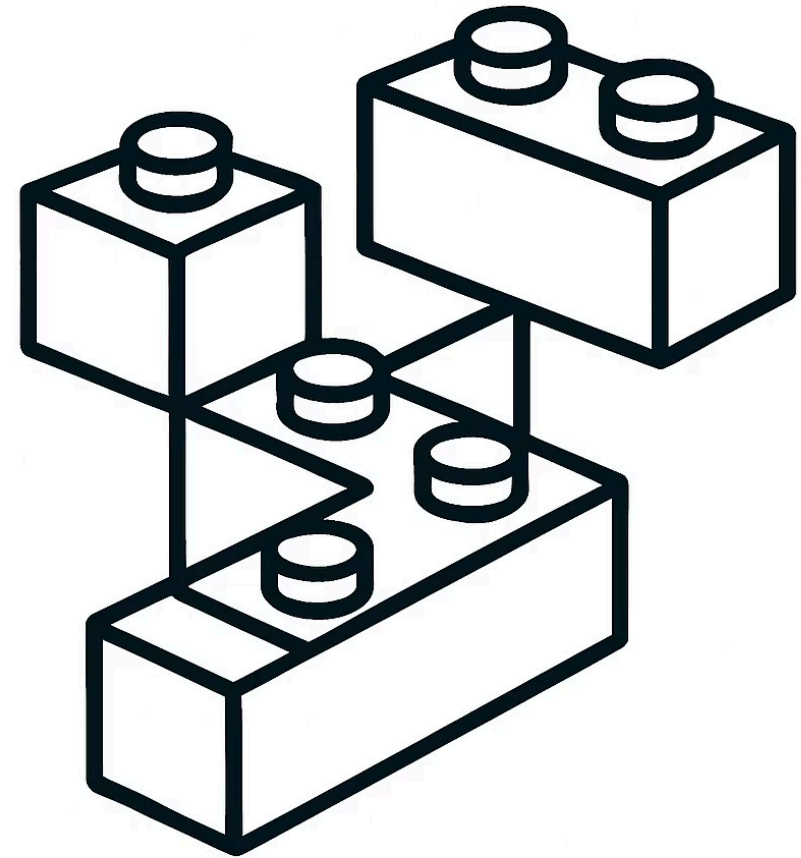
```
const tipoColor: Record<TipoPokemon, string> = {  
  normal: '#A8A878',  
  fuego: '#F08030',  
  agua: '#6890F0',  
  planta: '#78C850',  
  electrico: '#F8D030',  
  // ... los 18 tipos  
};  
  
// Uso: color dinámico garantizado por TypeScript  
const color = tipoColor[tipo]; // siempre existe
```

# Composición de componentes

La composición es el principio de construir componentes complejos a partir de otros más simples. En React esto se traduce en **átomos** (componentes mínimos) que se combinan en **moléculas** (componentes compuestos).

```
// Átomo — componente mínimo e independiente
function PokemonTypeBadge({ tipo }: { tipo: TipoPokemon }) {
  return (
    <View style={[styles.badge, { backgroundColor: tipoColor[tipo] }]}>
      <Text>{tipo}</Text>
    </View>
  );
}

// Molécula — combina átomos
function PokemonCard({ nombre, tipos }: PokemonCardProps) {
  return (
    <View>
      <Text>{nombre}</Text>
      {tipos.map(t => <PokemonTypeBadge key={t} tipo={t} />)}
    </View>
  );
}
```



# PokeApp — PokemonTypeBadge.tsx

Este es el componente completo tal y como aparece en la aplicación de ejemplo. Observa cómo el estilo del badge se construye dinámicamente a partir del diccionario `tipoColor`, y cómo los estilos estáticos se mantienen en `StyleSheet.create` por rendimiento.

```
export default function PokemonTypeBadge({
  tipo
}: PokemonTypeBadgeProps) {
  return (
    <View style={[styles.badge, { backgroundColor: tipoColor[tipo] }]}>
      <Text style={styles.label}>{tipo}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  badge: {
    borderRadius: 20,
    paddingHorizontal: 10,
    paddingVertical: 3,
  },
  label: {
    fontSize: 11,
    fontWeight: '600',
    color: '#fff',
    textTransform: 'capitalize',
  },
});
```

📌 **Buena práctica:** usa `StyleSheet.create` para los estilos estáticos y aplica estilos dinámicos en línea solo cuando dependan de props o estado.

# Props opcionales y valores por defecto

## Definición del tipo

```
type BadgeProps = {  
  tipo: TipoPokemon;  
  size?: 'sm' | 'md'; // opcional  
};
```

El operador `?` indica que la prop es opcional. TypeScript infiere que su valor puede ser `'sm' | 'md' | undefined`.

## Desestructuración con default

```
function PokemonTypeBadge({  
  tipo,  
  size = 'md' // valor por defecto  
}: BadgeProps) {  
  const fontSize = size === 'sm'  
    ? 10  
    : 13;  
  return (  
    <View style={ [  
      styles.badge,  
      { backgroundColor: tipoColor[tipo] }  
    ]}>  
      <Text style={ [  
        styles.label,  
        { fontSize }  
      ]}>{tipo}</Text>  
    </View>  
  );  
}
```

- ❗ Asignar el valor por defecto directamente en la desestructuración es el patrón recomendado en React moderno – evita comprobaciones `if (size === undefined)` dentro del cuerpo del componente.

# Tour starter/PokemonTypeBadge.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica con dos puntos marcados como `TODO` que deberás completar durante el ejercicio. El tipo unión solo incluye 'normal' y el color es un gris estático – tu misión es expandirlos.

```
// TODO: amplía la unión con los 18 tipos
type TipoPokemon = 'normal';

type PokemonTypeBadgeProps = {
  tipo: TipoPokemon;
};

export default function PokemonTypeBadge({
  tipo
}: PokemonTypeBadgeProps) {
  // TODO: obtén el color del diccionario tipoColor
  const color = '#BDBDBD';

  return (
    <View style={[styles.badge, { backgroundColor: color }]}>
      <Text style={styles.label}>{tipo}</Text>
    </View>
  );
}
```

⚠ No modifiques los estilos base del `StyleSheet` – céntrate en los dos `TODO` marcados arriba.

✓ SOLUCIÓN

# Solución — solucion/PokemonTypeBadge.tsx

La solución completa añade el diccionario `tipoColor` tipado con `Record<TipoPokemon, string>` y conecta el color dinámicamente al badge. Compara tu implementación con esta referencia.

```
const tipoColor: Record<TipoPokemon, string> = {
  normal: '#A8A878',
  fuego: '#F08030',
  agua: '#6890F0',
  planta: '#78C850',
  electrico: '#F8D030',
  // ... los 18 tipos completos
};

export default function PokemonTypeBadge({
  tipo
}: PokemonTypeBadgeProps) {
  return (
    <View style={[
      styles.badge,
      { backgroundColor: tipoColor[tipo] }
    ]}>
      <Text style={styles.label}>{tipo}</Text>
    </View>
  );
}
```

✓ Si TypeScript no muestra ningún error y el badge cambia de color según el tipo, ¡lo has conseguido! 🎉

# Ejercicio T03

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el patrón de props opcionales.

## 1 Abre el fichero de inicio

Navega a `starter/PokemonTypeBadge.tsx` en tu editor.

## 2 Amplía la unión de tipos

Añade los 18 tipos Pokémon a `TipoPokemon`.

## 3 Crea el diccionario de colores

Implementa `tipoColor: Record<TipoPokemon, string>` con un color hexadecimal para cada tipo.

## 4 Conecta el color al badge

Sustituye el color estático por `tipoColor[tipo]` en el estilo del componente.

## 5 Bonus: prop `size`?

Añade `size?: 'sm' | 'md'` al tipo de props y ajusta el `fontSize` dinámicamente.

# Resumen de la sesión

Estos son los cinco conceptos clave que has trabajado hoy. Dominarlos te permitirá escribir componentes React robustos, mantenibles y completamente tipados con TypeScript.

| Concepto                        | Sintaxis clave                                 | Para qué sirve                             |
|---------------------------------|------------------------------------------------|--------------------------------------------|
| Componente funcional            | <code>function X(props: T) { }</code>          | Unidad mínima de UI reutilizable           |
| Tipos de props                  | <code>type XProps = { ... }</code>             | Contrato de la API del componente          |
| Union types                     | <code>'a'   'b'   'c'</code>                   | Restringe a valores válidos conocidos      |
| <code>Record&lt;K, V&gt;</code> | <code>Record&lt;TipoPokemon, string&gt;</code> | Diccionario tipado y exhaustivo            |
| Props opcionales                | <code>prop?: tipo + valor default</code>       | Flexibilidad sin sacrificar seguridad      |
| Composición                     | Átomos → Moléculas                             | Construir UI compleja desde piezas simples |

✅ ¡Enhorabuena! Ahora tienes las herramientas para crear componentes tipados, reutilizables y bien compuestos en cualquier proyecto React + TypeScript. 🚀